מפרט CR-027: אפליקציית הנייד, רישום מכשירים, נוכחות וחיישנים, ודחיפה דרך ממסר SmplWise

מסמך	מפרט CR-027: אפליקציית הנייד, רישום מכשירים, נוכחות וחיישנים, ודחיפה דרך ממסר SmplWise
נתיב המקור	docs/changes/CR-027-MOBILE-PRESENCE-PUSH.md
גרסת המקור	origin/main @ b6eb212ec
הוכן לחבילה	2026-10-24
מצב	הנוסח המקורי נשמר ללא שינוי; עיצוב בלבד. טיוטה לבדיקת עורך דין, אינו ייעוץ משפטי.

מפרט שינוי (אנגלית): איך נוכחות עובדים וחיישני מכשיר נשלחים למערכת, מה נשמר, והדחיפה דרך ממסר של SmplWise.

CR-027 — The phone app: device registration, presence and sensor sharing, push through the SmplWise relay

Numbering: registered as CR-027 on 2026-10-05 (next free after CR-026). Branch `pilot/mobile-presence-push-server` (from `origin/main` at 2.0.0). Migration **0057** (`0056` is taken by `integ/201`).

Status: server side implemented in two phases on the branch (section 10 says what each phase built, cut and tested). The app side is built by the Codex session on the owner's Mac against `docs/api/mobile-presence-contract.md` (the final contract; it supersedes the drafts in `mobile/ios-shell/HANDOFF_IOS_CODEX.md` section 6 and `HANDOFF_IOS_ADDENDUM_SENSORS_PUSH.md` sections A.4 / B.4 where they differ, section 9 lists every difference).

Owner decisions (2026-10-04 / 2026-10-05) adopted: continuous location; presence published but **off until switched on** with an orderly employee notice first; **(a)** the SmplWise push relay is approved; **(b)** sensors v1 = every sensor of the addendum's catalogue, each one chosen by the user; **(c)** an administrator may **require** sensors (e.g. location) as a condition for using the system from the phone app - a user who has not switched a required sensor on is not let in until they do.

Related: CR-008 (remote channel, Web Push `services/push.py`), CR-012 (Android push options - this CR picks the relay pattern it describes as "HA Companion style"), CR-018 (notification core: the app is one more delivery channel), `docs/security/HA_IDENTITY_RBAC_HE.md` (identity: the device belongs to an HA user; a device token is never an HA token).

1. What it does

Part	Built	Where
Device registration	A signed-in user names their phone; the server answers a 256-bit **device token shown once** and keeps only its SHA-256. One row per (user, phone install); ten devices per user; the same install registering again rotates the token.	`services/presence.py`, `routers/presence.py`, table `mobile_devices`
Config for the app	`GET presence/config`: master switch, cadence, the employee notice (text + version), the allowed sensors with their catalogue and intervals, geofences, beacons, Wi-Fi sites, the required-sensors summary. While the switch is off every allow-list is empty (the second gate).	same

Part	Built	Where
Employee notice	`presence.notice_text` + `presence.notice_version`. The version rises by itself when the text changes **or a sensor is newly allowed**; a device must `ack` the current version before any event batch is accepted.	same
Events	Batches of `fix / enter / exit / sensor` events (<= 50, body <= 64 KiB, idempotent per `client_event_id`, validated, sensor allow-list enforced with `400 sensor_not_allowed`), with the phone's `status` (permission state, sharing switch, per-sensor on/off). The server derives the **current presence state per device** (inside / site / last fix) and keeps a retention-limited log.	table `mobile_presence_events`, `presence_json` on the device row
Manage	rename (device token or owner), unregister (device token, owner, or system.configure), the caller's own list, the administrator's list of everyone's devices (`presence.sensors.view`).	same
Required-sensors policy	`presence.required_sensors` (section 4): the server refuses (403 `presence_required`) every request of a blocked app session except the routes that lead out of that state; `GET /me` carries `presence_gate` so the shell shows one clean state; administrators are never blocked; a **break-glass** suspends the policy for a while with a reason.	`services/presence.gate_for / enforce`, `auth._principal`, `routers/me.py`, `shell/sw-app.ts`
Settings UI	the switch, allowed sensors, notice, :(system.configure) **אפליקציה לנייד** › הגדרות retention, the policy with its break-glass, the registered devices. Minimal operator-screen rules, Hebrew, no infrastructure branding.	`frontend/src/screens/system-presence.ts`, `api/presence.ts`
Retention	`presence.retention_days` (default 30, 1–365): the event log is pruned by the hourly janitor; expired app push messages and revoked device rows (30 days) go with it.	`services/presence.janitor`, `main.py` housekeeping
Push (phase 2)	The app registers its **relay token** with the server; the server is one more CR-018 channel (`app`) that calls the SmplWise relay with a generic payload; the app's notification extension fetches the real text with its device token. The relay itself is in `services/push-relay/` (section 6).	`services/mobile_push.py`, `routers/mobile_notifications.py`, table `mobile_push_messages`, `services/push-relay/`

2. Identity and credentials

- A **device token** (`arxd_` + 43 base64url characters) is the phone's credential after registration. The server stores `sha256(token)`; the token appears in exactly one response (the registration) and in no log, audit row or list. Lookup is one indexed read; unknown tokens cost a counter (20 per token key per minute, 300 per installation per minute, then 429) so that guessing stays noisy and cheap for us. A revoked device's hash is replaced, so the token dies at once.
- A device token is **not** an HA token and never meets the remote channel's bearer path: the presence routes resolve it themselves and open their own connection, so an unauthenticated call never takes the SQLite write gate (the same rule CR-008 applies to sessions). On the remote channel the body cap for such a caller is the anonymous 64 KiB.
- The device belongs to the **user who registered it** (`user_id` of the Arx session at that moment). A user without any binding cannot register (403, audited). Every route that names a device checks ownership: another user's device answers like a missing one (404) and the attempt is audited `presence.device.access / denied`.
- Permissions: `presence.report` (every default role but kiosk - a kiosk display is not a person) registers and reports one's own phone; `presence.sensors.view` (system_admin) sees everyone's devices; the

settings are `system.configure`. Both catalogues (`roles.json`, `contracts/examples/role-catalog.design.json`) and `PERMISSION_LABELS` carry them.

3. Data and privacy

- Stored per device: name, platform, app / OS version, model, the token hash, the last reported `status` (permission state, sharing switch, per-sensor on/off), the **current** presence state (`inside`, `site_id`, the last coordinate with its accuracy, source, time), push registration data (section 6), counters and timestamps.
- The event log holds derived values only: one coordinate per fix, enter / exit with a site id, a flat sensor value object (<= 20 keys, <= 1 KiB). No address lookup, no raw streams, no history on the phone beyond its unsent queue.
- Retention is a setting (`presence.retention_days`, default 30). Unregistering deletes the device's events at once.
- Audit actions: `presence.device.register / rename / unregister / access`, `presence.notice.ack`, `presence.settings.update` (the notice's length and version, never its text), `presence.break_glass`, `presence.gate.refused` (one row per user per 10 minutes), plus the permission refusals. No coordinate is audited.
- The notice wording is the owner's / counsel's (`mobile/ios-shell/EMPLOYEE_NOTICE_DRAFT_HE.md` is a draft). The server ships an empty text; the UI shows a placeholder until the owner writes it.

4. The required-sensors policy (owner decision c, 2026-10-05)

`presence.required_sensors` (one JSON setting, edited in אפליקציה לנייד › הגדרות or `PUT presence/settings`):

Field	Meaning	Default
`enabled`	the policy is on	false
`sensors`	the sensors a user must share (each must be in `sensors_allowed`)	[]
`apply_to_web`	also refuse a plain browser / PWA session of a covered user (who may have no app at all)	false
`max_stale_hours`	how recent the phone's `status` report must be to count (1 h – 30 d)	48
`roles`, `users`	scope: when either is non-empty the policy covers only users holding one of these roles (any scope) or named here; both empty = everyone	[]
`exempt_users`	never covered	[]

Evaluation (`services/presence.gate_for`) for a user on a channel:

1. Inert while `presence.enabled` is off (nothing can be reported, so nothing is required) - `reason: presence_disabled`.
2. **Never** an installation administrator (`is_system_admin`) or an exempt user - `reason: exempt`.
3. Out of scope by roles / users - `reason: not_in_scope`. A browser session when `apply_to_web` is off - `reason: web_not_covered`.
4. Otherwise a sensor counts as shared when **some** live device of the user reported it `on` within `max_stale_hours`, acknowledged the **current** notice version, and - for location - also reports `sharing:`

`true` and a usable permission (`always` / `when_in_use`). `missing` = required minus shared; `blocked` = missing is not empty.

5. A **break-glass** (`POST presence/settings/break-glass {hours, reason}`, system.configure, audited) sets `blocked: false` for everyone while it lasts; `reason: break_glass`; `hours: 0` ends it.

Enforcement: `auth._principal` runs `presence.enforce` on every authenticated request. A session **of the app** (user agent contains `SmplWiseArx/` - both shells send it) of a blocked user gets `403 presence_required` with `details.missing`, except on `/me`, `/presence/*`, `/notifications/devices*`, `/notifications/app/*`, `/notifications/categories`, `/auth/*`, `/health`. `GET /me` carries `presence_gate` and the shell shows one state ("נדרש להפעיל שיתוף נתונים … מיקום"); the app's native sensor screen is where the user switches the sensor on, after which the next `/me` is open again. The policy never changes a permission, so nothing else needs re-checking.

Employment-law note. Requiring location (or any sensor) as a condition for using a work tool is a condition of employment in Israel's terms (Privacy Protection Law; the employer's duty of proportionality and notice). The employee notice of section 1 is the instrument that covers it: it must say which sensors are required, for what purpose, that the requirement applies only when using the system from the phone app (or also the browser, when so configured), how to switch sharing off and what follows. The owner / counsel writes that text; this CR only makes the requirement visible, auditable (every refusal is an audit row per user per window; every policy change and break-glass is audited with the actor) and reversible (one switch, the break-glass, the exempt list).

5. Settings (`presence.*` rows of `settings`, served and edited by `GET/PUT presence/settings`)

`enabled` (false), `mode` (continuous), `interval_s` (60, 30–3600), `distance_filter_m` (50, 10–5000), `notice_text`, `notice_version` (1, rises by itself), `sensors_allowed` (all eight), `intervals_s` (battery 900, steps 300, app_state 300), `sites` (<= 20 geofences `{id, name, lat, lon, radius_m}`), `beacons`, `wifi_sites`, `retention_days` (30), `required_sensors` (section 4), `break_glass` (read-only; written by its own route). The UI edits the switch, the allowed sensors, the notice, retention and the policy; geofences, beacons, Wi-Fi sites and intervals are API-only in this phase (section 10).

6. Push (phase 2)

Architecture (addendum B.2, approved): APNs / FCM only accept messages signed with the **publisher's** key, so a customer server cannot push to the SmplWise app directly and must not hold our Apple key. A small **SmplWise push relay** (`services/push-relay/`, a Cloudflare Worker in TypeScript) holds the APNs auth key and the FCM service account as **runtime secrets only** (never in git), stores nothing but `sha256(relay_token) → {platform, push token}`, and accepts `POST /v1/push` from Arx servers with a generic payload. The app registers its APNs / FCM token with the relay, gets an opaque relay token, and registers that with each Arx server (`POST notifications/devices`, device token). The server's `app` channel (CR-018 `services/notify_channels` interface) writes one `mobile_push_messages` row per device with the real title / body at the administrator's lock-screen level, and calls the relay with `{relay_token, category, notification_id, priority}` only. The app's Notification Service Extension fetches `GET notifications/app/{id}` with its device token (single-use in spirit, 24 h expiry) and shows the text; if that fails the generic text stays. No sensitive text ever travels through Apple, Google or the relay.

Server side: `notify_policy` accepts `app` as a v1 channel (default off per source; a chip per source in הגדרות › התראות), quiet hours follow the matrix's **push** column (no column of its own), the per-device mute list

(`PATCH notifications/devices/{id} {muted: [...]}`) is honoured at planning, the relay's 404 / 410 unregisters the device's push, 429 / 5xx retry with the notifier's heap, outcomes land in the delivery log like every channel. Configuration: add-on options / env `SW_PUSH_RELAY_URL` and `SW_PUSH_RELAY_KEY` (the server's key at the relay); without them the channel plans `skipped / channel_unavailable`. Tests use a fake relay (httpx MockTransport) and `services/push-relay/fake_relay.py` is a tiny local relay for the app developer.

7. Security review notes

- Device tokens: hashed at rest, shown once, prefix-separated from HA tokens, revocation immediate, guess-throttled.
- Cross-user: every device route resolves ownership from the token or the session; a device token acts on its own device only (even an administrator's token); admin overrides are explicit permissions (`system.configure` to remove, `presence.sensors.view` to see), never implied by the HA admin flag.
- Bodies: 64 KiB for the app on the remote channel (anonymous tier), 1 MiB locally; 50 events per batch; 1 KiB per sensor value; 8000 characters of notice; 20 geofences. Pydantic rejects unknown shapes; the service validates values.
- Rate limits: registrations 10 / hour / user; event batches 60 / minute / device; settings writes 20 / minute; token-guess counters as above.
- The policy can never lock out an administrator and never changes permissions; a refusal is audited, not silent.
- Push: generic payloads only; the relay holds no text; the server holds the relay token like a Web Push endpoint (a capability useless without the server's relay key); the extension's fetch is authenticated by the device token and limited to that device's own messages.

8. Tests

`smplwise_vms/backend/tests/test_presence.py` (phase 1): roles and labels; registration (token shape, hash only, rotation, own list without tokens, validation, uniqueness with suggestion, permission refusal audited, 10 / hour rate, 10 devices); config and the switch (empty allow-lists while off; the notice version rising on text change and newly allowed sensors, not on trimming; audit without the text; system.configure only; validation); ack and events gating on the switch and the notice; events (idempotency, time-ordered storage, state from the newest location event, geofence matching, every validation code, the batch cap); security (device token on another device, another user's session, administrator's list and remove, token dies on revoke, audit rows; token guessing 429; 413 on an oversized body; 400 on an oversized value; per-device rate); rename; the required-sensors policy end to end (block / unblock by status, sharing off, stale report, new notice version, break-glass with reason and audit, roles / users scope, apply_to_web, exempt, inert while off, cleared); retention janitor. Phase 2: `test_mobile_push.py` - registration validation, categories, mutes, unregister; the channel's generic payload (no text, no ids but the message id, the server key as bearer) and the device's text fetch (own device only, 24 h expiry, the generic lock-screen level); policy off / reach / mute / critical / quiet hours; the relay's 429 retry on the notifier heap with the re-check, 410 dropping the registration, the unconfigured relay; the test push and its rate; unregistering a device drops its messages. The relay Worker is typechecked (`tsc --noEmit -p services/push-relay`), NOT run against APNs / FCM (needs the owner's keys; README section 1). `frontend/tests/evidence-presence-settings.spec.ts` (desktop / tablet / mobile, mocked backend): defaults, the save patch, the notice version, the policy problem hints, break-glass, the shell's gate state.

9. Differences from the drafts in the iOS handoff (what Codex must read)

1. `GET notifications/{notification_id}` for the extension is **`GET notifications/app/{message_id}`**: the session route owns `notifications/{nid}` and the app's message ids are per-device, not inbox ids.
2. `POST presence/devices` answers `created: true|false` next to the token; `GET presence/config` with a device token adds `device: {device_id, name, notice_ack_version}` and `required_sensors: {enabled, sensors}`; the sensor catalogue is served under `sensors.catalog`.
3. Events are refused as a whole (`400 sensor_not_allowed` names the sensor in `details.sensor`; `409 presence_disabled`; `409 notice_ack_required` with the current version); the answer carries `site_id` and `notice_version` besides `accepted / duplicates / inside`. `inside` is `null` when no geofence is configured and no enter / exit was reported.
4. `status` may be sent alone (`events: []`) - it is how "sharing off" and permission changes arrive, and it is what the required-sensors policy reads.
5. `DELETE presence/devices/{id}` answers 204; the row is kept 30 days (revoked) for the audit trail.
6. `/me` carries `presence_gate`; a blocked app session gets `403 presence_required` on every other route (section 4).
7. `GET notifications/categories` answers the CR-018 categories (`safety` is `critical`); `PATCH notifications/devices/{id}` takes `{muted: [...]}` of those ids.

10. What each phase built, what was cut and why

Phase 1 (this branch, first milestone): everything in sections 1–5 and 7–8 except push. Cut: an Arx screen that shows who is in the building (`presence.view`, "later" in the handoff); geofence / beacon / Wi-Fi-site editors in the UI (API only; the owner has no sites with coordinates yet and the catalogue's `sites` table has none - adding coordinates to sites is a separate change); a per-user "my devices" card in the account page (the app shows "המכשיר הזה"; the web card waits for the frontend bridge changes listed in the handoff section 10). Phase 2 (second milestone): section 6.

10a. WHOIN034-295 (board r034 / r035, target 2.16.0): "מי במבנה" and the place editors

Branch `pilot/WHOIN034-295` from `origin/main` (v2.14.0). **No migration** (settings rows and the existing event log). Presence stays **OFF by default per installation** (the employee notice is not final): while `presence.enabled` is off the screen shows one clean "off" state and the API answers nothing but the floor list.

Part	Built	Where
Permission	`presence.view` "צפייה במי שנמצא במבנה": system_admin only by default, **sensitive** (a custom role names it among its sensitive permissions), scoped like the map - a floor-scoped holder sees the people placed on that floor; a person placed on a site only needs installation scope. Both role catalogues and `PERMISSION_LABELS`.	`roles.json`, `contracts/examples/role-catalog.design.json`, `routers/access.py`

Part	Built	Where
Read API	`GET presence/people` (session; presence.view at any scope, refusal audited): per person the most precise fresh placement - **beacon** (`beacon` value `beacon_id` or `uuid` + `major` / `minor`, at `min_proximity` or nearer, at most `beacon_max_age_minutes` old) > **Wi-Fi** (`network` value `ssid_hash` matched against a Wi-Fi site, at most `wifi_max_age_minutes` old) > **geofence** (`inside: true` at most `stale_minutes` old); a newer exit wins; several phones: the most precise, then the newest. Counts per floor and area, per site, totals. Audited `presence.people.view` once per viewer per 10 minutes with counts, never names.	`services/presence_people.py`, `routers/presence.py`
Privacy	A device counts only while it acknowledged the **current** notice version and its phone reports the sensor on (location also `sharing: true`). Per-person **opt-out** (`GET/PUT presence/people/me`, self-service, presence.report, audited `presence.people.opt_out`; nobody can set it for someone else): never named, never placed on a floor or area, counted only in `totals.hidden`. `people.show_names: false` = counts only. The administrator sees how many opted out, not who.	same
Thresholds	`presence.people {stale_minutes 30 (5-1440), beacon_max_age_minutes 10 (1-240), wifi_max_age_minutes 15 (1-240), min_proximity near, show_names true, refresh_seconds 30 (10-300)}`; a partial save keeps the other keys.	`services/presence.py`, הגדרות › אפליקציה לנייד › מסך מי במבנה
Place editors	Beacons `{id, name, uuid, major?, minor?, site_id, floor_id?, zone_id?}` (UUID format, a configured geofence, the area on that floor, unique id and (uuid, major, minor)); Wi-Fi sites `{id, name, site_id, floor_id?, ssid_hash_salt, ssid_hashes, add_ssids}` - `add_ssids` is write-only, the server hashes `sha256_hex(salt + ssid)` and keeps only hashes (salt made per site, kept on later saves); geofences get a server id when sent without one. A geofence still named by a beacon / Wi-Fi site cannot be removed alone. **One save is validated as a whole before anything is written** (a refusal commits its audit row, so a half-applied save was possible before). Audit carries counts only.	`services/presence.py _validate_places`
Screen	`#/explore/presence`, tab for presence.view holders): KPI row (inside / on a floor /) **מי במבנה** › מפה site only / asked not to appear), the floor's plan with each area labelled by its count, the list with search; fits the viewport (the list scrolls inside its panel; a phone switches between map and list); off / no places / counts only / forbidden states; polled per `refresh_seconds`. Built to the infrastructure .(screens' look (no approved presence mockup existed in `private/review`	`frontend/src/screens/explore-presence.ts` (+ `-logic.ts`), `i18n/presence.ts`
Settings	has three pages (`?tab=mobile&page=places|screen`): the existing sharing / הגדרות › אפליקציה לנייד ;geofences, beacons and Wi-Fi sites on one page, three cards - **מקומות** ;policy page - **מסך מי במבנה** .the thresholds and the opt-out count	`system-presence.ts`, `system-presence-places.ts` (+ `-logic.ts`)
Opt-out card	On the personal settings page (#/system/notifications), only while sharing is on and the user holds presence.report.	`screens/presence-opt-out.ts`
Contract	additive: beacon / Wi-Fi config keys, the SSID hash scheme, and the `beacon` / `network` value spellings the server reads (contract sections 2 and 4). The app (Codex, r037) must send `beacon_id` (or `uuid` / `major` / `minor`) and `proximity`, and `ssid_hash`.	`docs/api/mobile-presence-contract.md`

Tests: `tests/test_whoin034_295.py`; `frontend/tests/unit-whoin034-295.spec.ts`, `frontend/tests/evidence-whoin034-295.spec.ts` (screenshots under `docs/evidence/WHOIN034-295/`), `unit-tabs-config.spec.ts` updated for the new map tab. Not built: a live dot per person on the plan (areas carry counts, which is also the privacy-safer choice), history of presence (the time-scrub change request owns it), an Arabic / Russian column for the new words (they fall back to Hebrew).

11. Rollback

Migration 0057 is additive (three new tables). Running the previous version ignores them; the new permissions in the role catalogues are harmless to older code. The relay is external: removing `SW_PUSH_RELAY_URL` disables the channel.

תקציר בעברית

מה נבנה (צד השרת של האפליקציה לנייד):

1. **רישום מכשיר** – המשתמש נותן שם לטלפון באפליקציה; השרת מחזיר מפתח מכשיר פעם אחת ושומר רק גיבוב שלו. עד 10 מכשירים למשתמש; רישום חוזר של אותו טלפון מחליף את המפתח.
2. **מתג ראשי כבוי כברירת מחדל** – כל עוד השיתוף כבוי, הטלפונים לא אוספים דבר ולא מבקשים הרשאות. ההפעלה בהגדרות › **אפליקציה לנייד**, יחד עם רשימת החיישנים המאושרים (שמונה: מיקום, פעילות, צעדים, גובה, סוללה, רשת, קרבה, פעילות האפליקציה – כל אחד נבחר גם על ידי העובד), ההודעה לעובדים ומשך שמירת היומן.
3. **הודעה לעובדים עם גרסה** – שינוי בטקסט, או אישור חיישן חדש, מעלה את הגרסה; כל מכשיר חייב לאשר את הגרסה הנוכחית לפני שהוא שולח נתונים. הנוסח המשפטי הוא של בעל המערכת (השרת מגיע עם טקסט ריק).
4. **דיווחים** – מנות של עד 50 אירועים (מיקום, כניסה/יציאה, ערכי חיישן), חסינות לכפילויות, עם מצב הטלפון (הרשאות, מתג השיתוף, אילו חיישנים דולקים). השרת מחזיק מצב נוכחות נוכחי לכל מכשיר ויומן מוגבל בזמן (ברירת מחדל 30 יום).
5. **מדיניות חיישנים נדרשים** (החלטת 05-10-2026) – מנהל יכול לדרוש חיישנים (למשל מיקום) כתנאי לשימוש מהאפליקציה. משתמש שלא הפעיל חיישן נדרש לא נכנס מהאפליקציה עד שיפעיל; המסך מציג סיבה ברורה. מנהלי מערכת לעולם לא נחסמים; דפדפן רגיל לא נפגע אלא אם סומן "גם בדפדפן"; יש השעיה זמנית (עם סיבה, מתועדת) ורשימת פטורים; המדיניות רדומה כשהשיתוף כבוי. **הערה משפטית:** דרישה כזו היא תנאי העסקה לעניין חוק הגנת הפרטיות – ההודעה לעובדים חייבת לכסות אותה (אילו חיישנים, לאיזו מטרה, מתי חלה, איך מכבים ומה קורה אז).
6. **התראות דחיפה (שלב 2)** – דרך ממסר SmplWise קטן (מאושר) שמחזיק את מפתחות Apple/Google רק כסודות בזמן ריצה; השרת שולח אליו רק מזהה וקטגוריה, והאפליקציה מושכת את הטקסט האמיתי מהשרת עם מפתח המכשיר.

מה לא נבנה בשלב זה ולמה: מסך "מי במבנה" (סומן "בהמשך" במסמך המסירה); עריכת גדרות גאוגרפיות/משדרים/רשתות במסך (זמין ב-API בלבד – לאתרים עדיין אין קואורדינטות בקטלוג); כרטיס "המכשירים שלי" בדף החשבון (מחכה לשינויי הגשר בחזית).